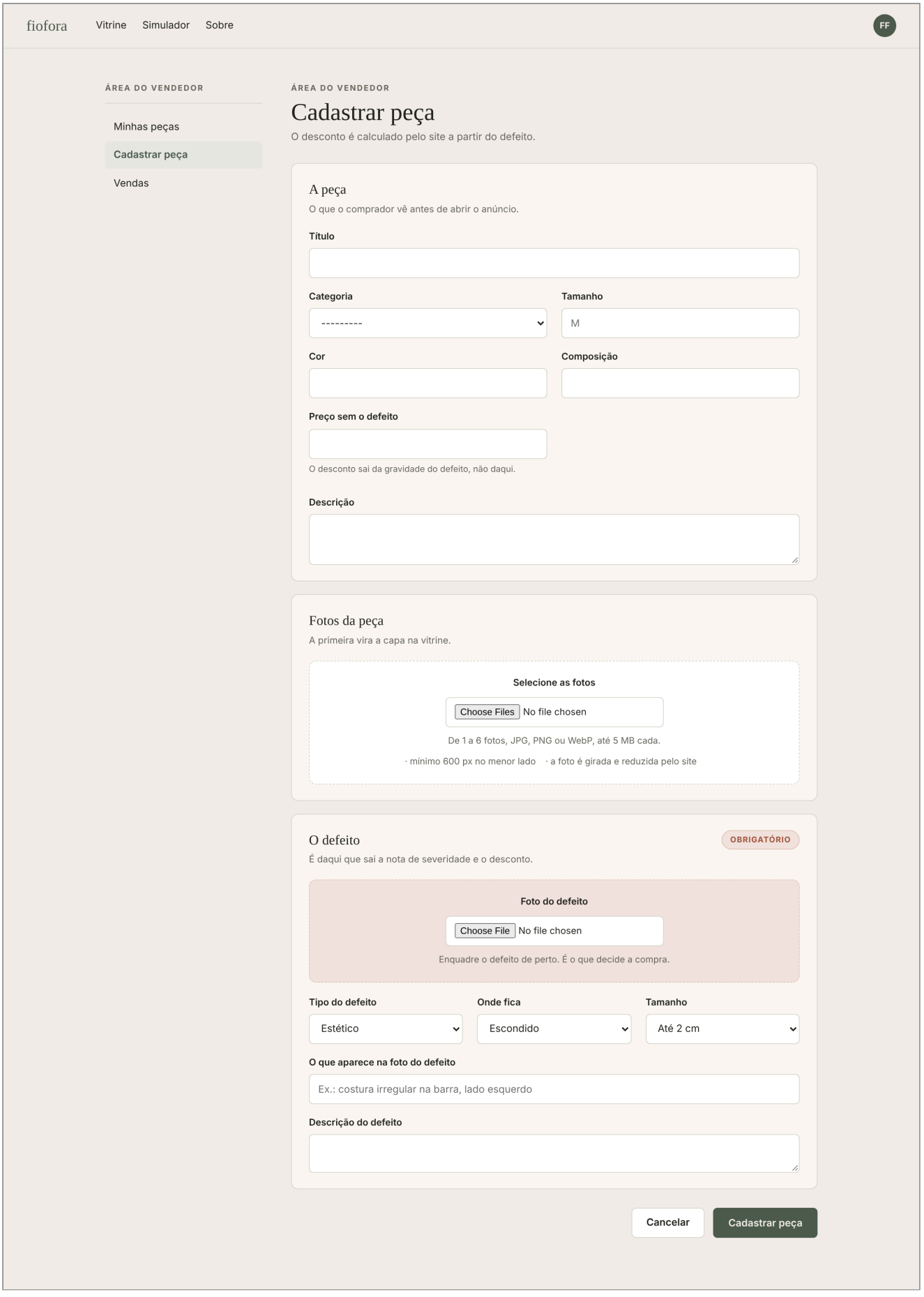
FioFora — Projeto Integrado (Software) · Sprint 7
Membro 4 — Antonio de Azambuja Fehring

Envio das fotos da peça e do defeito no cadastro

Tarefa difícil · validação, vínculo ao produto e armazenamento de mídia.

O FioFora vende roupa com defeito. Um anúncio que descreve o defeito em texto mas não mostra pede que a pessoa acredite na palavra do vendedor, que foi justamente o que as entrevistas da Sprint 2 apontaram como o motivo de ninguém confiar em brechó online. Por isso a foto do defeito é obrigatória aqui, e peça sem ela não entra na vitrine.

A tela



Cadastro em `/pecas/cadastrar/`, rodando em Django 5.2.

O que é verificado em cada arquivo

Regra	Por quê
É mesmo uma imagem	O arquivo é aberto com o Pillow. A extensão e o content-type são escritos pelo navegador e qualquer um troca; um arquivo qualquer renomeado para .jpg passa pela extensão e morre aqui.
JPG, PNG ou WebP	Formato que todo navegador exibe. O teste confere que GIF e os outros são recusados.
Até 5 MB	Foto de celular moderno passa disso com facilidade e o upload trava em conexão ruim.
Mínimo 600 px no menor lado	Abaixo disso o defeito vira um borrão no zoom, e o anúncio deixa de cumprir o que promete.
De 1 a 6 fotos da peça	Uma para existir na vitrine; seis porque acima disso ninguém olha.
Foto do defeito obrigatória	É a razão de ser do anúncio.

O que acontece com a imagem depois de aprovada

Ela é girada conforme o EXIF, porque foto de celular chega deitada e o defeito apareceria de lado. É reduzida a 2000 px no maior lado, para não guardar 12 megapixels de uma imagem que vai ser exibida com 800. E os metadados são descartados: foto de celular carrega a coordenada de GPS de onde foi tirada, que normalmente é a casa do vendedor, e isso não pode ir para a internet junto com o anúncio.

O arquivo é salvo em `media/pecas/<id da peça>/<nome sorteado>`. O nome que o vendedor deu não entra no caminho: além de poder vazar informação, dois vendedores mandando "frente.jpg" se sobrescreveriam.

A gravação da peça, do defeito e das fotos acontece dentro de uma transação. Ou entra tudo, ou não entra nada. Peça gravada sem a foto do defeito é exatamente o estado que o site não pode ter.

Testes

```
$ python manage.py test catalogo.tests.CadastroCompleto catalogo.tests.ArquivosRecusados \
      catalogo.tests.PreparoDaImagem catalogo.tests.ArquivoNoDisco

Found 19 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
...................
----------------------------------------------------------------------
Ran 19 tests in 6.218s

OK
```

Os testes geram imagens de verdade com o Pillow em vez de usar arquivos falsos, então a validação é exercitada do mesmo jeito que seria com uma foto real. Entre o que cobrem: arquivo de texto renomeado para .jpg, formato fora da lista, arquivo acima do limite, imagem pequena demais, fotos demais, nenhuma foto, cadastro sem foto do defeito, erros de várias fotos chegando juntos, redução da imagem grande, descarte dos metadados, nome do arquivo que não vai para o caminho, e remoção do arquivo do disco ao apagar a foto.

Como ligar no projeto

Em `settings.py`:

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

E, no `urls.py` do projeto, servir os arquivos enviados enquanto é desenvolvimento:

```
urlpatterns = [
    ...
    path("pecas/", include("catalogo.urls")),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

O projeto passa a depender do Pillow (`pip install pillow`), que é o que o `ImageField` do Django exige. A pasta `media/` não entra no Git.

catalogo/imagens.py

```
"""Validação e preparo das imagens enviadas no cadastro de peça.

Fica num módulo separado porque a validação não é do formulário: o admin, um
futuro import em lote e os testes precisam da mesma regra.

A checagem é feita abrindo a imagem com o Pillow, não olhando a extensão nem
o content-type que o navegador manda. Os dois são escritos pelo cliente e
qualquer um troca. Um .exe renomeado para .jpg passa pela extensão e morre
aqui.
"""
from io import BytesIO

from django.core.exceptions import ValidationError
from django.core.files.uploadedfile import InMemoryUploadedFile
from PIL import Image, ImageOps, UnidentifiedImageError

FORMATOS = {"JPEG": "jpg", "PNG": "png", "WEBP": "webp"}
EXTENSOES = "JPG, PNG ou WebP"

TAMANHO_MAX = 5 * 1024 * 1024          # 5 MB
LADO_MINIMO = 600                       # px no menor lado
LADO_MAXIMO = 2000                      # px no maior lado, depois de reduzir
QUALIDADE = 85

MAX_FOTOS_PECA = 6
MIN_FOTOS_PECA = 1


class Imagem:
    """O resultado de validar um upload: o arquivo pronto e seus dados."""

    def __init__(self, arquivo, formato, largura, altura, bytes_):
        self.arquivo = arquivo
        self.formato = formato
        self.largura = largura
        self.altura = altura
        self.bytes = bytes_


def validar_e_preparar(upload, rotulo="A imagem"):
    """Valida um arquivo enviado e devolve a versão que vai para o disco.

    Levanta ValidationError com uma mensagem que faz sentido para o vendedor,
```

```
    não com o erro técnico. Quem está cadastrando não sabe o que é EXIF.
    """
    if upload.size > TAMANHO_MAX:
        raise ValidationError(
            f"{rotulo} tem {upload.size / (1024 * 1024):.1f} MB. "
            f"O limite é {TAMANHO_MAX // (1024 * 1024)} MB."
        )

    try:
        imagem = Image.open(upload)
        imagem.verify()           # detecta arquivo truncado ou que não é imagem
        upload.seek(0)
        imagem = Image.open(upload)
    except (UnidentifiedImageError, OSError):
        raise ValidationError(
            f"{rotulo} não pôde ser lida. Envie um arquivo {EXTENSOES}."
        )

    if imagem.format not in FORMATOS:
        raise ValidationError(
            f"{rotulo} está em {imagem.format or 'formato desconhecido'}. "
            f"Use {EXTENSOES}."
        )

    if min(imagem.size) < LADO_MINIMO:
        raise ValidationError(
            f"{rotulo} tem {imagem.width}×{imagem.height} px. "
            f"O menor lado precisa ter pelo menos {LADO_MINIMO} px, "
            "senão o defeito não aparece no zoom."
        )

    return _preparar(imagem, upload.name)


def _preparar(imagem, nome):
    """Gira pelo EXIF, reduz se for grande demais e tira os metadados.

    Girar importa porque foto de celular vem deitada e o defeito aparece de
    lado. Tirar metadados importa porque foto de celular carrega a coordenada
    de GPS de onde foi tirada, que normalmente é a casa do vendedor.
    """
    imagem = ImageOps.exif_transpose(imagem)
    formato = imagem.format or "JPEG"

    if max(imagem.size) > LADO_MAXIMO:
        imagem.thumbnail((LADO_MAXIMO, LADO_MAXIMO), Image.LANCZOS)

    if formato == "JPEG" and imagem.mode not in ("RGB", "L"):
        imagem = imagem.convert("RGB")

    limpa = Image.new(imagem.mode, imagem.size)
    limpa.putdata(list(imagem.getdata()))

    buffer = BytesIO()
    if formato == "JPEG":
        limpa.save(buffer, "JPEG", quality=QUALIDADE, optimize=True)
    elif formato == "PNG":
        limpa.save(buffer, "PNG", optimize=True)
    else:
        limpa.save(buffer, "WEBP", quality=QUALIDADE)

    tamanho = buffer.tell()
    buffer.seek(0)

    base = nome.rsplit(".", 1)[0][:40] or "foto"
    arquivo = InMemoryUploadedFile(
        buffer,
        field_name="arquivo",
        name=f"{base}.{FORMATOS[formato]}",
        content_type=f"image/{FORMATOS[formato]}",
```

```
        size=tamanho,
        charset=None,
    )
    return Imagem(arquivo, formato, limpa.width, limpa.height, tamanho)


def validar_lote(uploads, rotulo="As fotos da peça"):
    """Valida a lista de fotos da peça e devolve todas preparadas.

    Junta os erros em vez de parar no primeiro: quem mandou seis fotos quer
    saber quais deram problema de uma vez, não uma por tentativa.
    """
    uploads = list(uploads)

    if len(uploads) < MIN_FOTOS_PECA:
        raise ValidationError(f"Envie pelo menos {MIN_FOTOS_PECA} foto da peça.")
    if len(uploads) > MAX_FOTOS_PECA:
        raise ValidationError(
            f"{rotulo}: são {len(uploads)} arquivos e o limite é {MAX_FOTOS_PECA}."
        )

    preparadas, erros = [], []
    for posicao, upload in enumerate(uploads, start=1):
        try:
            preparadas.append(validar_e_preparar(upload, f"A foto {posicao}"))
        except ValidationError as erro:
            erros.extend(erro.messages)

    if erros:
        raise ValidationError(erros)
    return preparadas
```

catalogo/forms.py

```
"""Formulário de cadastro de peça com envio de fotos."""
from django import forms

from . import imagens
from .models import Defeito, Foto, Peca


class EntradaMultipla(forms.ClearableFileInput):
    """Input de arquivo que aceita vários de uma vez.

    O Django 5 deixou de permitir multiple no ClearableFileInput padrão, então
    o widget e o campo abaixo são a receita da própria documentação.
    """

    allow_multiple_selected = True


class CampoDeVariosArquivos(forms.FileField):
    def __init__(self, *args, **kwargs):
        kwargs.setdefault("widget", EntradaMultipla(attrs={"accept": "image/*"}))
        super().__init__(*args, **kwargs)

    def clean(self, data, initial=None):
        limpar_um = super().clean
        if isinstance(data, (list, tuple)):
            return [limpar_um(d, initial) for d in data]
        return [limpar_um(data, initial)] if data else []


class CadastroDePecaForm(forms.ModelForm):
    """Cadastro de peça: dados, defeito e fotos numa tela só."""

    fotos_peca = CampoDeVariosArquivos(
        label="Fotos da peça",
```

```
        required=False,
        help_text=(
            f"De {imagens.MIN_FOTOS_PECA} a {imagens.MAX_FOTOS_PECA} fotos, "
            f"{imagens.EXTENSOES}, até {imagens.TAMANHO_MAX // (1024 * 1024)} MB cada."
        ),
    )
    foto_defeito = forms.ImageField(
        label="Foto do defeito",
        required=False,
        widget=forms.ClearableFileInput(attrs={"accept": "image/*"}),
        help_text="Enquadre o defeito de perto. É o que decide a compra.",
    )
    legenda_defeito = forms.CharField(
        label="O que aparece na foto do defeito",
        max_length=140,
        required=False,
        widget=forms.TextInput(
            attrs={"placeholder": "Ex.: costura irregular na barra, lado esquerdo"}
        ),
    )

    tipo = forms.TypedChoiceField(
        label="Tipo do defeito", choices=Defeito.TIPOS, coerce=int
    )
    localizacao = forms.TypedChoiceField(
        label="Onde fica", choices=Defeito.LOCALIZACOES, coerce=int
    )
    extensao = forms.TypedChoiceField(
        label="Tamanho", choices=Defeito.EXTENSOES, coerce=int
    )
    descricao_defeito = forms.CharField(
        label="Descrição do defeito",
        required=False,
        widget=forms.Textarea(attrs={"rows": 2}),
    )

    class Meta:
        model = Peca
        fields = [
            "titulo", "categoria", "tamanho", "cor",
            "composicao", "preco_original", "descricao",
        ]
        widgets = {
            "descricao": forms.Textarea(attrs={"rows": 3}),
            "preco_original": forms.NumberInput(attrs={"step": "0.01", "min": "0"}),
            "tamanho": forms.TextInput(attrs={"placeholder": "M"}),
        }
        labels = {"preco_original": "Preço sem o defeito"}
        help_texts = {
            "preco_original": "O desconto sai da gravidade do defeito, não daqui.",
        }

    # ---------------------------- validação ----------------------------

    def clean_fotos_peca(self):
        """Valida as fotos da peça abrindo cada arquivo."""
        uploads = self.cleaned_data.get("fotos_peca") or []
        self._fotos_peca = imagens.validar_lote(uploads)
        return uploads

    def clean_foto_defeito(self):
        """A foto do defeito é obrigatória: é a razão de ser do anúncio."""
        upload = self.cleaned_data.get("foto_defeito")
        if not upload:
            raise forms.ValidationError(
                "A foto do defeito é obrigatória. Sem ela a peça não pode "
                "ser publicada."
            )
        self._foto_defeito = imagens.validar_e_preparar(upload, "A foto do defeito")
        return upload
```

```
    # ------------------------------ gravação ------------------------------

    def save(self, vendedor, commit=True):
        """Grava peça, defeito e fotos.

        A view chama isto dentro de uma transação. Ou entra tudo, ou não entra
        nada: peça sem foto do defeito no banco é exatamente o estado que o
        site não pode ter.
        """
        peca = super().save(commit=False)
        peca.vendedor = vendedor
        peca.save()

        Defeito.objects.create(
            peca=peca,
            tipo=self.cleaned_data["tipo"],
            localizacao=self.cleaned_data["localizacao"],
            extensao=self.cleaned_data["extensao"],
            descricao=self.cleaned_data.get("descricao_defeito", ""),
        )

        for ordem, imagem in enumerate(getattr(self, "_fotos_peca", [])):
            self._criar_foto(peca, imagem, Foto.PECA, ordem)

        defeito = getattr(self, "_foto_defeito", None)
        if defeito:
            self._criar_foto(
                peca, defeito, Foto.DEFEITO, 0,
                legenda=self.cleaned_data.get("legenda_defeito", ""),
            )

        if peca.pode_publicar:
            from django.utils import timezone

            peca.status = "disponivel"
            peca.publicada_em = timezone.now()
            peca.save(update_fields=["status", "publicada_em"])

        return peca

    @staticmethod
    def _criar_foto(peca, imagem, papel, ordem, legenda=""):
        foto = Foto(
            peca=peca, papel=papel, ordem=ordem, legenda=legenda,
            formato=imagem.formato, largura=imagem.largura,
            altura=imagem.altura, bytes=imagem.bytes,
        )
        foto.arquivo.save(imagem.arquivo.name, imagem.arquivo, save=False)
        foto.save()
        return foto
```

catalogo/models.py (model Foto)

```
def caminho_da_foto(instancia, nome_original):
    """Onde o arquivo vai parar dentro de MEDIA_ROOT.

    Uma pasta por peça, nome sorteado. O nome que o vendedor deu ao arquivo
    não entra no caminho: além de poder vazar coisa (IMG_casa_joao.jpg), dois
    vendedores mandando "frente.jpg" se sobrescreveriam.
    """
    extensao = {"JPEG": "jpg", "PNG": "png", "WEBP": "webp"}.get(
        instancia.formato, "jpg"
    )
    return f"pecas/{instancia.peca_id}/{uuid.uuid4().hex}.{extensao}"


class Foto(models.Model):
```

```
    """Uma imagem de uma peça.

    Guardar as fotos numa tabela à parte, e não em colunas foto_1, foto_2 da
    Peca, é o que permite ter um número variável delas e saber qual mostra o
    defeito sem olhar o nome do arquivo.
    """

    PECA = "peca"
    DEFEITO = "defeito"
    PAPEIS = [
        (PECA, "Peça"),
        (DEFEITO, "Defeito"),
    ]

    peca = models.ForeignKey(Peca, related_name="fotos", on_delete=models.CASCADE)
    arquivo = models.ImageField("arquivo", upload_to=caminho_da_foto)
    papel = models.CharField(max_length=10, choices=PAPEIS, default=PECA)
    ordem = models.PositiveSmallIntegerField(default=0)
    legenda = models.CharField(max_length=140, blank=True)
    formato = models.CharField(max_length=8, blank=True)
    largura = models.PositiveIntegerField(null=True, blank=True)
    altura = models.PositiveIntegerField(null=True, blank=True)
    bytes = models.PositiveIntegerField(null=True, blank=True)
    enviada_em = models.DateTimeField(auto_now_add=True)

    class Meta:
        # Defeito primeiro quando se pede tudo junto, depois a ordem escolhida.
        ordering = ["papel", "ordem", "id"]

    def __str__(self):
        return f"{self.get_papel_display()} de {self.peca.titulo}"

    @property
    def e_do_defeito(self):
        return self.papel == self.DEFEITO

    @property
    def peso_legivel(self):
        """O tamanho do arquivo em KB ou MB, para mostrar na tela."""
        if not self.bytes:
            return ""
        if self.bytes < 1024 * 1024:
            return f"{self.bytes / 1024:.0f} KB"
        return f"{self.bytes / (1024 * 1024):.1f} MB"

    def delete(self, *args, **kwargs):
        """Apagar o registro apaga o arquivo junto.

        Sem isso o disco vai enchendo de imagem órfã que ninguém mais
        consegue alcançar pelo site.
        """
        arquivo = self.arquivo
        super().delete(*args, **kwargs)
        arquivo.delete(save=False)
```

catalogo/views.py (cadastro)

```
"""Views do catálogo — cadastro com fotos e galeria do vendedor."""
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.db import transaction
from django.shortcuts import get_object_or_404, redirect, render

from .forms import CadastroDePecaForm
from .models import Foto, Peca


@login_required
```

```
def peca_cadastrar(request):
    """Cadastro de peça, com as fotos da peça e a foto do defeito."""
    if request.method == "POST":
        form = CadastroDePecaForm(request.POST, request.FILES)
        if form.is_valid():
            with transaction.atomic():
                peca = form.save(vendedor=request.user)
            messages.success(
                request,
                f"{peca.titulo} cadastrada com "
                f"{peca.fotos.count()} foto(s).",
            )
            return redirect("catalogo:minhas_pecas")
    else:
        form = CadastroDePecaForm()

    return render(
        request,
        "catalogo/peca_cadastrar.html",
        {"form": form, "secao_ativa": "cadastrar"},
    )
```

templates/catalogo/peca_cadastrar.html

```
{% extends "catalogo/vendedor_base.html" %}

{# Cadastro de peça com envio de fotos. Sprint 7, Membro 4. #}

{% block titulo %}Cadastrar peça · FioFora{% endblock %}

{% block vendedor_conteudo %}

  <div class="cabecalho-conteudo">
    <div>
      <p class="sobrelinha">Área do vendedor</p>
      <h1 class="titulo">Cadastrar peça</h1>
      <p class="subtitulo">O desconto é calculado pelo site a partir do defeito.</p>
    </div>
  </div>

  <form method="post" enctype="multipart/form-data" novalidate>
    {% csrf_token %}

    {% if form.non_field_errors %}
      <div class="aviso aviso--alerta">
        <p class="aviso__titulo">Não foi possível cadastrar</p>
        {% for erro in form.non_field_errors %}<p>{{ erro }}</p>{% endfor %}
      </div>
    {% endif %}

    <section class="cartao">
      <div class="cartao__cabecalho">
        <div>
          <h2 class="cartao__titulo">A peça</h2>
          <p class="cartao__apoio">O que o comprador vê antes de abrir o anúncio.</p>
        </div>
      </div>
      <div class="grade">
        {% include "catalogo/_campo.html" with campo=form.titulo largo=1 %}
        {% include "catalogo/_campo.html" with campo=form.categoria %}
        {% include "catalogo/_campo.html" with campo=form.tamanho %}
        {% include "catalogo/_campo.html" with campo=form.cor %}
        {% include "catalogo/_campo.html" with campo=form.composicao %}
        {% include "catalogo/_campo.html" with campo=form.preco_original %}
        {% include "catalogo/_campo.html" with campo=form.descricao largo=1 %}
      </div>
    </section>
```

```
    <section class="cartao">
      <div class="cartao__cabecalho">
        <div>
          <h2 class="cartao__titulo">Fotos da peça</h2>
          <p class="cartao__apoio">A primeira vira a capa na vitrine.</p>
        </div>
      </div>

      <div class="campo campo--largo {% if form.fotos_peca.errors %}campo--erro{% endif %}">
        <div class="zona">
          <p class="zona__rotulo">Selecione as fotos</p>
          {{ form.fotos_peca }}
          <p class="zona__dica">{{ form.fotos_peca.help_text }}</p>
          <ul class="regras">
            <li>mínimo 600 px no menor lado</li>
            <li>a foto é girada e reduzida pelo site</li>
          </ul>
        </div>
        {% for erro in form.fotos_peca.errors %}
          <p class="campo__erro">{{ erro }}</p>
        {% endfor %}
      </div>
    </section>

    <section class="cartao">
      <div class="cartao__cabecalho">
        <div>
          <h2 class="cartao__titulo">O defeito</h2>
          <p class="cartao__apoio">É daqui que sai a nota de severidade e o desconto.</p>
        </div>
        <span class="selo selo--alerta">Obrigatório</span>
      </div>

      <div class="campo campo--largo {% if form.foto_defeito.errors %}campo--erro{% endif %}"
           style="margin-bottom:18px">
        <div class="zona zona--defeito">
          <p class="zona__rotulo">Foto do defeito</p>
          {{ form.foto_defeito }}
          <p class="zona__dica">{{ form.foto_defeito.help_text }}</p>
        </div>
        {% for erro in form.foto_defeito.errors %}
          <p class="campo__erro">{{ erro }}</p>
        {% endfor %}
      </div>

      <div class="grade grade--tres">
        {% include "catalogo/_campo.html" with campo=form.tipo %}
        {% include "catalogo/_campo.html" with campo=form.localizacao %}
        {% include "catalogo/_campo.html" with campo=form.extensao %}
      </div>

      <div class="grade" style="margin-top:16px">
        {% include "catalogo/_campo.html" with campo=form.legenda_defeito largo=1 %}
        {% include "catalogo/_campo.html" with campo=form.descricao_defeito largo=1 %}
      </div>
    </section>

    <div class="acoes">
      <a class="botao botao--secundario botao--alto" href="{% url 'catalogo:minhas_pecas'
%}">Cancelar</a>
      <button class="botao botao--primario botao--alto" type="submit">Cadastrar peça</button>
    </div>
  </form>

{% endblock %}
```

templates/catalogo/_campo.html

```
{# Um campo do formulário: rótulo, controle, ajuda e erros. #}
<div class="campo {% if largo %}campo--largo{% endif %} {% if campo.errors %}campo--erro{% endif %}">
  <label for="{{ campo.id_for_label }}">{{ campo.label }}</label>
  {{ campo }}
  {% if campo.help_text %}<p class="campo__ajuda">{{ campo.help_text }}</p>{% endif %}
  {% for erro in campo.errors %}<p class="campo__erro">{{ erro }}</p>{% endfor %}
</div>
```